

SmartRoadSense System

An IoT-based Road Defect Detection and Analysis System

Project Report

1. Project Title

SmartRoadSense System: An IoT-based Road Defect Detection and Analysis System.

SmartRoadSense is a crowd-sensing Internet of Things (IoT) system designed to automatically detect road defects - such as potholes and other surface irregularities - using vehicle-mounted sensors, and to report them to a cloud platform for monitoring and maintenance prioritization.

2. Introduction

The Internet of Things (IoT) has become a foundational technology for building smarter, more responsive urban infrastructure. By embedding low-cost sensors and connectivity into everyday physical assets, cities and governments can move from periodic, manual inspection cycles toward continuous, data-driven monitoring. Road infrastructure is a particularly strong candidate for this transformation: roads are used constantly, their condition changes gradually and then suddenly (through potholes and surface failures), and the consequences of delayed maintenance - vehicle damage, traffic accidents, and increased long-term repair costs - are significant and well documented in public infrastructure management.

This project, SmartRoadSense, applies IoT principles to road quality monitoring. Rather than relying on dedicated survey vehicles or citizen complaints, the system is designed to run on vehicles that already travel the road network as part of their normal duties - specifically government vehicles such as police patrol cars and municipality vehicles. Each vehicle is equipped with a small, low-cost embedded computer (a Raspberry Pi) connected to a motion sensor and a GPS module. As the vehicle drives, the system continuously analyzes vibration data to detect abnormal events consistent with potholes or road defects, and reports these events - together with location and speed - to a cloud-based IoT platform (ThingsBoard) over the lightweight MQTT protocol.

The remainder of this report describes the problem this project addresses, the objectives guiding its design, the system architecture and hardware/software components used, the implementation methodology, the results obtained during testing, the challenges encountered along the way, and the conclusions and future directions for the project.

3. Problem Statement

Road authorities in most regions currently rely on a reactive maintenance model: repair work is scheduled only after a citizen files a complaint, after a scheduled (and infrequent) manual road survey, or - in the worst case - only after a vehicle is damaged or a traffic accident occurs as a direct result of a road defect. This approach has several well-known shortcomings.

First, response times are slow: a pothole may exist for weeks or months before it is noticed and reported through official channels. Second, manual road surveys are expensive and labor-intensive, requiring dedicated personnel and vehicles to physically traverse the road network, which limits how frequently such surveys can realistically be performed. Third, and most importantly, this reactive model means that the people who "discover" a defect are often the same people who suffer its consequences - a driver whose vehicle is damaged, or worse, a driver involved in an accident.

There is a clear need for a proactive, low-cost, and continuously operating mechanism that can detect road defects as they occur (or shortly after) and route that information to the relevant authorities automatically, without waiting for a complaint or a scheduled survey. This project addresses that need directly.

4. Objectives

The project pursues the following objectives:

- Design and implement an embedded system capable of detecting abnormal road-induced vibration in real time using low-cost, widely available hardware.
- Distinguish, at the edge (on the device itself), between normal driving vibration and vibration indicative of a genuine road defect, using a tiered classification (NORMAL / WARNING / DANGER).
- Provide immediate, human-visible feedback (LED indicators and an audible buzzer) reflecting the current road condition.
- Transmit location, speed, and vibration-severity data to a cloud IoT platform (ThingsBoard) over MQTT whenever a significant event is detected, without requiring constant, high-bandwidth data transmission.
- Design the system so that it can be deployed on government vehicles (police patrols, municipality vehicles) rather than private vehicles, avoiding privacy concerns while taking advantage of the wide, pre-existing road coverage these fleets already provide.
- Support two operating modes - a demonstration mode with simulated location/speed for presentations, and a real mode for deployment in an actual vehicle - toggled through a single configuration flag.
- Document the system's architecture, hardware wiring, and software logic clearly enough that it can be maintained, extended, and eventually integrated with a multi-vehicle verification mechanism on the platform side.

5. System Design / Architecture

SmartRoadSense follows a standard three-layer IoT architecture, separating the concerns of sensing/local decision-making, data transport, and cloud-side storage/analysis/visualization. This separation keeps each layer simple, testable, and replaceable independently of the others.

5.1 Edge Layer

The edge layer runs entirely on the Raspberry Pi 3 Model B installed in the vehicle. It is responsible for reading the motion sensor (MPU6050) and the GPS module (PA6H) at a high sampling rate, computing a road-condition classification locally, and driving the LED and buzzer outputs immediately - all without depending on network connectivity. This local-first design ensures the visual/audible feedback to the vehicle occupant works correctly even if the internet connection is temporarily unavailable.

5.2 Transport Layer

Communication between the edge device and the cloud platform uses MQTT, a lightweight publish/subscribe messaging protocol widely used in IoT systems. MQTT was chosen over heavier alternatives (such as plain HTTP requests) because it is designed for constrained devices and unreliable network links: it uses small message overhead, supports persistent connections, and offers configurable delivery guarantees through its Quality of Service (QoS) levels. This project uses QoS level 1, which guarantees that each published message is delivered at least once.

5.3 Cloud / Application Layer

The cloud layer is provided by ThingsBoard, an open-source IoT platform. ThingsBoard receives the MQTT messages published by the device, stores the contained values as time-series telemetry, and makes them available for visualization through customizable dashboards. ThingsBoard also provides a Rule Engine (Rule Chains) capable of triggering further logic - such as sending notifications - based on incoming telemetry, which is the intended mechanism for implementing multi-vehicle verification in a future iteration of this project (see Section 10, Challenges Encountered).

5.4 End-to-End Data Flow

- The MPU6050 is sampled approximately every 30 milliseconds to avoid missing short-duration shocks.
- The total acceleration magnitude and its deviation from the 1g baseline (gravity) are computed to represent net vibration.
- A rolling average smooths out sensor noise; a status (NORMAL / WARNING / DANGER) is computed by comparing the current metric against configured thresholds.
- LEDs and the buzzer are updated immediately based on this status, independent of network state.
- If the status is WARNING or DANGER, a JSON payload (location, speed, vibration data, status) is published to ThingsBoard over MQTT.
- ThingsBoard stores the telemetry and makes it available on a dashboard for authorities to monitor.

5.5 Non-Functional Considerations

Beyond the core functional flow described above, the design was guided by several non-functional considerations that shaped key decisions throughout the project:

- Reliability without connectivity: local vibration detection and actuator control must continue to function correctly even if the internet connection is lost, so these are never made dependent on a successful MQTT connection.
- Low bandwidth usage: telemetry is only published when a meaningful event occurs (WARNING/DANGER), not continuously, to minimize data usage across a large fleet of vehicles over cellular or shared connections.
- Low hardware cost: all components were selected to keep the per-vehicle cost low, since the system is intended to scale across an entire government fleet rather than a small number of dedicated survey vehicles.
- Privacy: by targeting government vehicles (police patrols, municipality vehicles) rather than private citizens' vehicles, the system avoids tracking personal movement data, while still achieving broad road coverage since these vehicles already patrol most of the Kingdom's roads as part of their routine duties.
- Configurability: nearly all tunable behavior (thresholds, timing, pin assignments) is centralized in a small number of configuration constants, so the system can be recalibrated without modifying its core logic.

6. Hardware and Software Components

6.1 Hardware Components

Component	Role in the System
Raspberry Pi 3 Model B	Central processing unit: reads sensors, runs detection logic, drives outputs, communicates over MQTT.
MPU6050	6-axis motion sensor (3-axis accelerometer + 3-axis gyroscope) used to measure vibration and tilt.
PA6H GPS Module	Provides geographic location (latitude/longitude) and ground speed via NMEA sentences over a serial (UART) connection.
3x LEDs (Green, Yellow, Red)	Provide immediate visual feedback of the current road condition (NORMAL / WARNING / DANGER).
Active Buzzer	Provides audible feedback during WARNING (intermittent tone) and DANGER (continuous tone) states.
3x 470 Ohm Resistors	Current-limiting resistors protecting each LED.
Breadboard and Jumper Wires	Used to physically connect all components to the Raspberry Pi's GPIO header.

6.2 Software and Libraries

Library / Tool	Purpose
Python 3	Primary programming language for the entire edge application.
RPi.GPIO	Controls the LED and buzzer outputs through the Raspberry Pi's GPIO pins.
smbus2	Provides I2C communication with the MPU6050 sensor.
pyserial / pynmea2	Read and parse NMEA sentences from the GPS module over the serial port.
paho-mqtt	Implements the MQTT client used to connect and publish telemetry to ThingsBoard.
ThingsBoard Cloud	Cloud IoT platform used for telemetry storage, dashboarding, and (future) rule-based alerting.

6.3 Wiring Diagram

The Raspberry Pi's physical pin numbering scheme (GPIO.BOARD) was used throughout, rather than the BCM numbering scheme, to make the wiring easier to verify visually against the physical header:

Component	Pin/Signal	Physical Pin Number
Shared breadboard ground	GND	Pin 9
Green LED (NORMAL)	Signal	Pin 11
Yellow LED (WARNING)	Signal	Pin 13
Red LED (DANGER)	Signal	Pin 15
Buzzer	Signal	Pin 12
MPU6050	VCC	Pin 1 (3.3V)
MPU6050	GND	Pin 6

Component	Pin/Signal	Physical Pin Number
MPU6050	SDA	Pin 3
MPU6050	SCL	Pin 5
GPS PA6H	VCC	Pin 4 (5V)
GPS PA6H	GND	Shared ground (Pin 9)
GPS PA6H	RX	Pin 8
GPS PA6H	TX	Pin 10

Note: the MPU6050's I2C address was verified in practice using the command `i2cdetect -y 1` and confirmed as 0x68.

7. Implementation Methodology

The system was developed incrementally, starting from a fixed configuration module and progressively adding sensor reading, output control, cloud communication, and finally the main control loop tying everything together. Each component was designed to be independently testable before integration.

7.1 Local Vibration Processing

The MPU6050 is sampled at a high rate (roughly every 30 milliseconds) so that brief, high-frequency shocks - such as a wheel striking a pothole - are not missed between readings. For each sample, the total acceleration magnitude is computed from the three axes, and the baseline of 1g (Earth's gravity at rest) is subtracted to obtain the 'net vibration' independent of the sensor's static orientation. A rolling average over the most recent samples is used to smooth out random sensor noise, and a short cooldown period (3 seconds) prevents a single, sustained shock event from being counted as multiple separate pothole events.

7.2 Demo Mode vs. Real Mode

Because simulating a real driving environment - realistic speed, GPS signal, and vibration - is impractical inside a classroom or presentation hall, the system was designed to support two operating modes toggled by a single configuration flag (DEMO_MODE), with the rest of the code adapting automatically:

Aspect	Demo Mode (DEMO_MODE = True)	Real Mode (DEMO_MODE = False)
Location	Fixed mock coordinates (a real location in Bahrain)	Live GPS reading
Speed	Fixed at 0, unused in calculations	Live GPS reading
Severity metric	Raw deviation from baseline (accel_delta_g)	Speed-normalized severity index
Thresholds	Tuned to react to a light hand movement on the breadboard	To be tuned after real-world driving trials

7.3 Speed-Normalized Severity (Real Mode)

In real driving conditions, the same physical pothole produces a larger shock the faster the vehicle is moving when it crosses it. Relying on raw acceleration alone would therefore over-report minor defects hit at high speed and under-report serious defects hit at low speed. To correct for this, a normalized severity index is computed as:

$$\text{severity_index} = \text{accel_delta_g} / \max(\text{speed_kmh}, 5)$$

A minimum speed floor of 5 km/h is applied to prevent this formula from producing artificially large values when the vehicle is nearly stationary, such as in traffic.

7.4 Publishing Strategy

The system is event-driven: no telemetry is published while the status is NORMAL, which avoids flooding the platform with unnecessary data. When the status becomes WARNING or DANGER, the event is published immediately; if the same status persists, the event is re-published every 5 seconds until the status changes. This keeps the platform informed of ongoing conditions without excessive message volume. In parallel, every reading - whether published or not - is logged to the terminal once per second for local monitoring during development and testing.

7.5 Code Organization

The final implementation is delivered as a single, well-commented Python file (main.py), organized into clearly labeled sections: configuration constants, the MPU6050 reader class, the GPS reader class (running on a background thread so it never blocks the main loop), the actuator (LED/buzzer) controller, the MQTT client wrapper, and the main control loop that ties all of the above together. A separate, fully simulated demo script (demo.py) was also developed to exercise the platform-side dashboard and telemetry pipeline using synthetic data following a realistic patrol route across multiple regions of Bahrain, without requiring any physical hardware.

7.6 Approximate Hardware Cost Breakdown

One of the design goals of this project was to keep the per-vehicle hardware cost low enough that the system could realistically be deployed across an entire government fleet. The table below summarizes the approximate cost of the components used per unit.

Component	Approx. Unit Cost (USD)
Raspberry Pi 3 Model B	19. 800 BHD
MPU6050 module	8 BHD
PA6H GPS module	6. 600 BHD
LEDs (x3), resistors, buzzer, breadboard, wires	4 BHD
Total (approx.)	38. 400 BHD

7.7 Testing Strategy

Testing was carried out incrementally, at three levels, to isolate faults quickly and avoid debugging the entire pipeline at once:

- Unit-level testing: each hardware component (MPU6050, GPS module, LEDs, buzzer) was tested in isolation using small standalone scripts before being integrated into the main application.
- Integration testing: the full local pipeline (sensor reading through to actuator control) was tested without network connectivity to confirm that local decision-making works independently of the cloud connection.
- End-to-end testing: the complete pipeline, including MQTT publishing and reception on the ThingsBoard platform, was tested by observing both the local terminal log and the ThingsBoard latest-telemetry view for each published event.

In addition, a fully simulated companion script (demo.py) was developed to generate synthetic - but realistic - telemetry along a patrol route spanning ten regions of Bahrain, without requiring any physical sensor hardware. This was used to independently exercise and validate the ThingsBoard-side dashboard and telemetry pipeline, decoupled from any hardware availability or GPS signal issues.

8. Results and Discussion

The system was tested primarily in a demonstration setting, simulating vibration by manually handling the breadboard rather than driving a real vehicle, due to the constraints of the testing environment (see Section 10). This still allowed the full data pipeline - sensor reading, local classification, actuator feedback, and MQTT publishing to ThingsBoard - to be validated end-to-end.

One notable result from testing was that simply holding the breadboard by hand, without any deliberate shaking, already produced a baseline vibration reading of approximately 0.09g due to natural hand tremor. This directly informed the threshold-tuning process: the initial WARNING/DANGER thresholds had to be raised above this baseline noise level to avoid constant false triggering while the device was simply being held, resulting in final demo thresholds of 0.18g for WARNING and 0.35g for DANGER.

A representative telemetry payload observed during testing, as published to the v1/devices/me/telemetry MQTT topic, is shown below:

```
{ "latitude": 26.0274, "longitude": 50.4999, "speed_kmh": 0.0, "total_accel_g": 1.282, "avg_vib_g": 0.114, "pitch_deg": -7.2, "roll_deg": -1.13, "status": "WARNING", "alert": true, "pothole_event": 0 }
```

This confirms that the full pipeline functions correctly: a locally detected vibration event is correctly classified, immediately reflected on the LED/buzzer outputs, and published as a well-formed JSON payload that ThingsBoard can ingest directly into its telemetry store for dashboard visualization.

The speed-normalized severity formula designed for real-mode operation was validated mathematically (by hand-checking sample calculations against the code's output) but not yet against real driving data, since outdoor GPS testing was still in progress at the time of writing (see Section 10). The real-mode WARNING/DANGER thresholds are therefore provisional and are expected to require recalibration once real driving trials are conducted.

9. Challenges Encountered

9.1 GPS Serial Communication Failure

During outdoor testing, the GPS module consistently reported no fix, even after several minutes with a clear view of the sky. Running gpsd in diagnostic mode revealed that the serial port was repeatedly 'hunting' across multiple baud rates and framing settings and failing to synchronize, which indicated that garbled (not simply absent) data was arriving on the serial line. This is a well-known issue on the Raspberry Pi 3: its full hardware UART (PL011) is bound to Bluetooth by default, leaving only the less reliable Mini-UART for GPIO serial communication unless Bluetooth is explicitly disabled and the primary UART is re-enabled through the boot configuration. This was identified and resolved by adjusting the relevant boot configuration settings.

9.2 Diagnosing a Suspected Motion Sensor Failure

At one point, the system began reporting a constant DANGER status with all acceleration values fixed at exactly zero - a result that is physically impossible for a working accelerometer, since gravity alone should always produce a non-zero reading. Reading the MPU6050's registers directly, bypassing the application code entirely, confirmed the same all-zero result, proving the fault was in the sensor connection rather than in the software logic. Reading the sensor's WHO_AM_I register (a fixed identification register that should always return 0x68) was used as a definitive hardware-level test to further isolate the fault to the physical wiring or power connection rather than the sensor's internal state.

9.3 Threshold Calibration Under Real-World Noise

Initial thresholds were selected without accounting for the natural noise floor produced simply by a person holding the device, which led to either constant false triggering (when thresholds were set too low) or a complete failure to trigger during deliberate demonstration shakes (when the metric being compared did not match the scale of the configured threshold). This was resolved by empirically observing baseline noise levels from live sensor logs and recalibrating both the metric definition and the threshold values accordingly.

9.4 Limited Testing Environment

Because outdoor, real-driving testing was not consistently available during development, most validation was performed indoors or in a stationary outdoor setting using the system's Demo Mode, with vibration simulated by hand rather than by an actual vehicle crossing a real pothole. This is sufficient to validate the data pipeline end-to-end but does not yet validate the real-mode thresholds under actual driving conditions.

9.5 Multi-Vehicle Verification Not Implemented

Although the system is conceptually designed with multi-vehicle verification in mind (requiring alerts from more than five independent vehicles at approximately the same location before an official maintenance alert is triggered), this feature was not implemented in the current version of the project. Realizing this logic requires a ThingsBoard Rule Chain capable of geofencing/clustering alerts from multiple devices, correlating them by location and vibration severity, and confirming a minimum vehicle count before escalating an alert - a level of configuration that goes beyond the platform's basic setup. More critically, validating that such logic actually works requires several independent devices (or access tokens) simultaneously reporting data from nearby locations, which was not achievable in the current testing environment since only a single device representing one vehicle was available. For this reason, multi-vehicle verification was excluded from the current project scope and is documented here as a known limitation and a candidate for future work, rather than being implemented without the ability to properly test and validate it.

10. Conclusion and Future Work

SmartRoadSense demonstrates a practical, low-cost approach to shifting road maintenance from a slow, reactive process toward a proactive, data-driven one. By combining local, real-time vibration classification with a lightweight, event-driven MQTT connection to a cloud IoT platform, the system is able to detect and report road defects using hardware that costs a small fraction of dedicated road-survey equipment, and can be deployed at scale on vehicles - such as government patrol and municipality vehicles - that already travel the road network as part of their normal operations.

The current implementation successfully validates the full data pipeline from sensor reading through to cloud telemetry storage, and includes a carefully designed demo mode that allows the system to be presented and tested without needing a live vehicle or GPS signal. Several important elements, however, remain as future work:

- Implementing and testing the multi-vehicle verification logic in a ThingsBoard Rule Chain, once multiple test devices become available.
- Calibrating the real-mode severity thresholds using data collected from actual driving trials.
- Maintaining a geo-tagged database of known, intentional speed bumps, so that recurring vibration signatures at those exact locations are automatically excluded from being flagged as road defects requiring maintenance.
- Exploring machine learning techniques to better classify the type of detected event (e.g., pothole vs. speed bump vs. generally rough road) beyond fixed thresholds.

Taken together, these next steps would move SmartRoadSense from a validated prototype toward a deployable, fleet-scale system capable of providing continuous, reliable, and actionable road-condition data to the relevant authorities.

11. References

- Raspberry Pi Foundation. Raspberry Pi 3 Model B Documentation. Available at: <https://www.raspberrypi.com/documentation/>
- TDK InvenSense. MPU-6050 Six-Axis (Gyro + Accelerometer) MEMS MotionTracking Device Product Specification.
- National Marine Electronics Association. NMEA 0183 Standard for Interfacing Marine Electronic Devices.
- OASIS. MQTT Version 3.1.1 / 5.0 Protocol Specification. Available at: <https://mqtt.org/>
- ThingsBoard. Official Documentation. Available at: <https://thingsboard.io/docs/>
- Eclipse Foundation. Eclipse Paho MQTT Python Client Documentation. Available at: <https://www.eclipse.org/paho/>

Appendix A: Key Configuration Parameters

For reference, the table below lists the main configurable parameters defined at the top of the system's source code (`road_quality_system.py`). Adjusting these values is the primary way to tune the system's behavior without modifying the underlying logic.

Parameter	Purpose	Value Used
DEMO_MODE	Toggles between simulated demo data and real GPS/speed data	True / False
MPU6050_I2C_ADDRESS	I2C address of the motion sensor	0x68
GPS_BAUD_RATE	Serial communication speed for the GPS module	9600
DEMO_WARNING_THRESHOLD_G	Vibration delta above which Demo Mode reports WARNING	0.18g
DEMO_DANGER_THRESHOLD_G	Vibration delta above which Demo Mode reports DANGER	0.35g
MIN_SPEED_FLOOR_KMH	Minimum speed used in the real-mode severity formula	5 km/h
REPEAT_INTERVAL_SECONDS	How often a sustained	5 seconds

Parameter	Purpose	Value Used
	WARNING/DANGER event is re-published	
POTHOLE_COOLDOWN_SECONDS	Minimum time between two counted pothole_event occurrences	3 seconds
LED_HOLD_SECONDS	Minimum time the yellow/red LED stays lit after a triggering event	2 seconds
MQTT_QOS	MQTT Quality of Service level used for publishing	1